



Performance Boundaries and Architectural Constraints of the BlueField-3

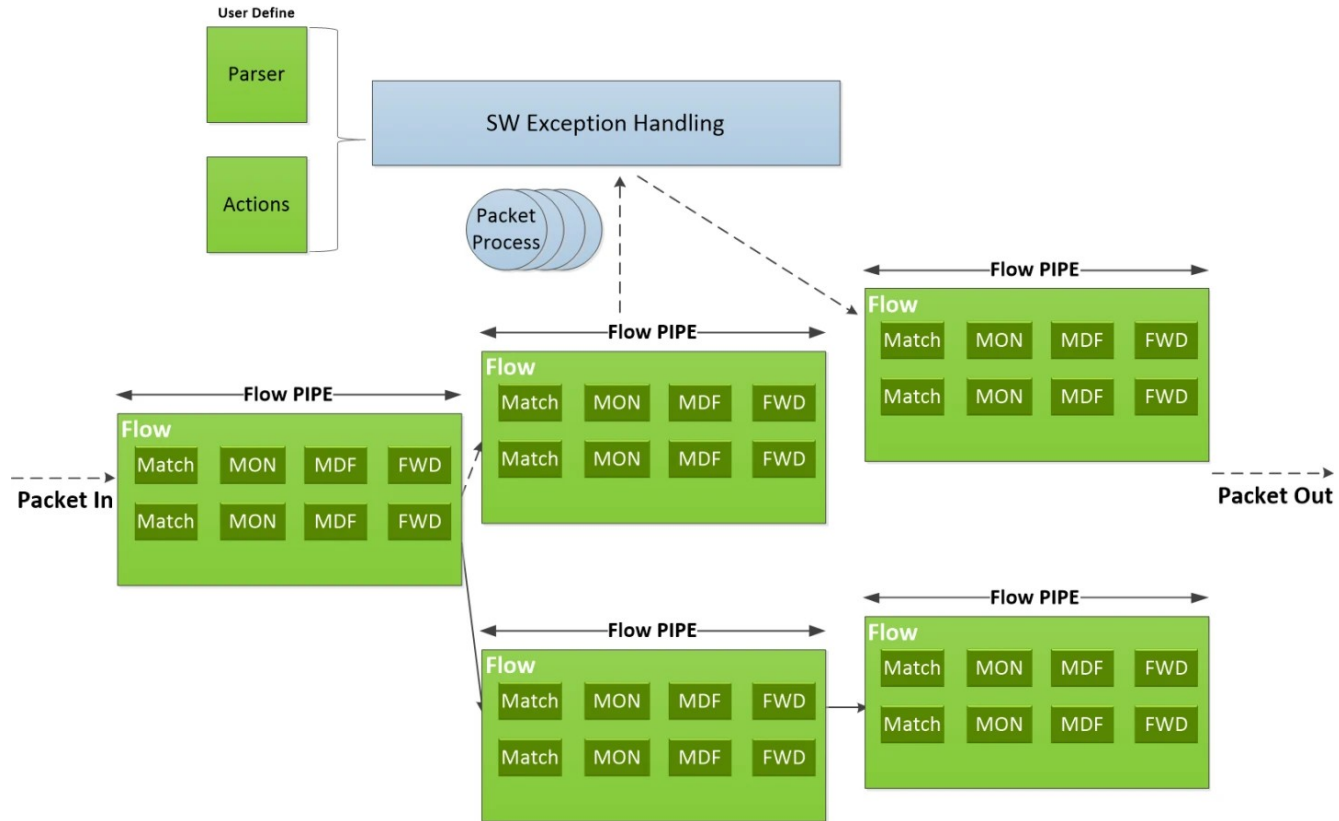
Sten Heimbrodt, Master Praktikumsbericht

Gliederung

- *Was bisher geschah...* und Refresher DOCA Flow
 - Pipe, Entry Semantik
 - Stand XenoFlow 0.1
- BlueField Architektur
- Experimente
 - ebpf_mac_rewrite
 - entry_latency
 - max_pipes
 - max_entries
 - hash_pipe
 - pcc
- Sonstiges und Ausblick

DOCA Flow

„DOCA Flow is the most fundamental API for building generic packet processing pipes in hardware. The DOCA Flow library provides an API for building a set of pipes, where each pipe consists of match criteria, monitoring, and a set of actions. Pipes can be chained so that after a pipe-defined action is executed, the packet may proceed to another pipe.“

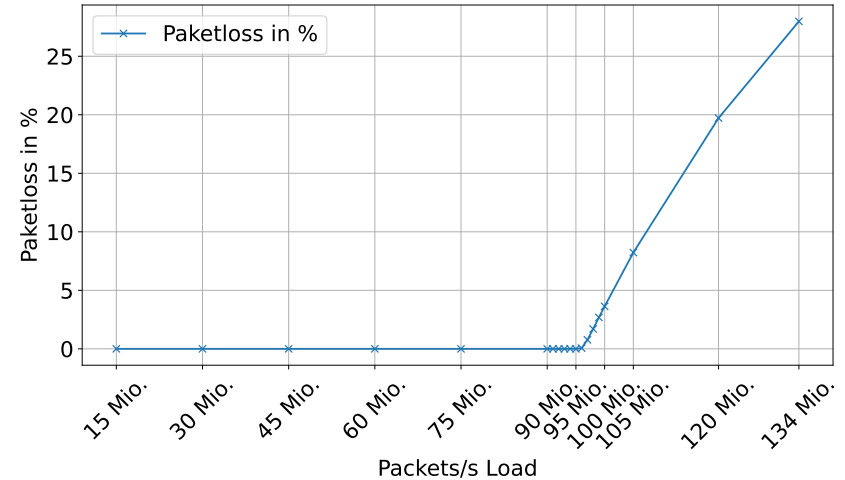
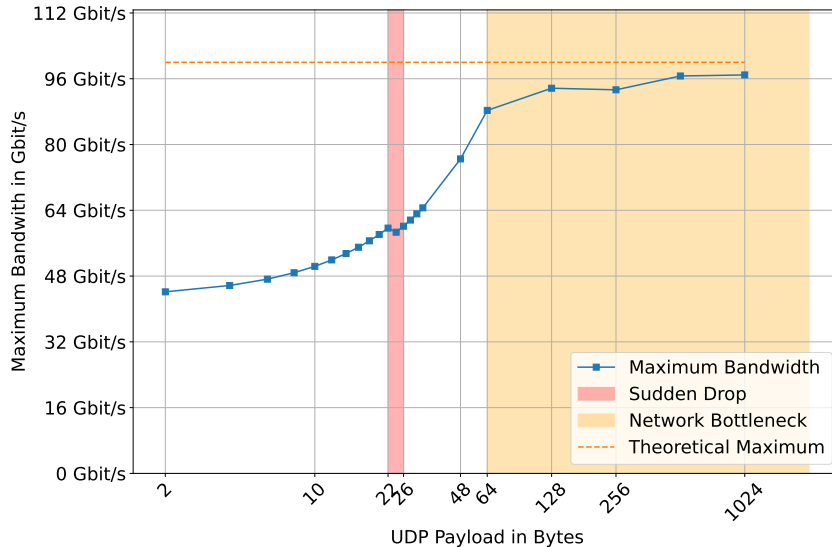


<https://docs.nvidia.com/doca/sdk/doca-flow/index.html>

Was bisher geschah...

- Implementierung eines Load-Balancers im Rahmen der Bachelorarbeit Q1 2025
- Verwendung von DOCA Flow als einzige Abhängigkeit
- Statischer L3 Load-Balancer mit 4-Tuple Match
- 95 Millionen Pakete pro Sekunde Verarbeitung
→ (98 Millionen bei size < 64)

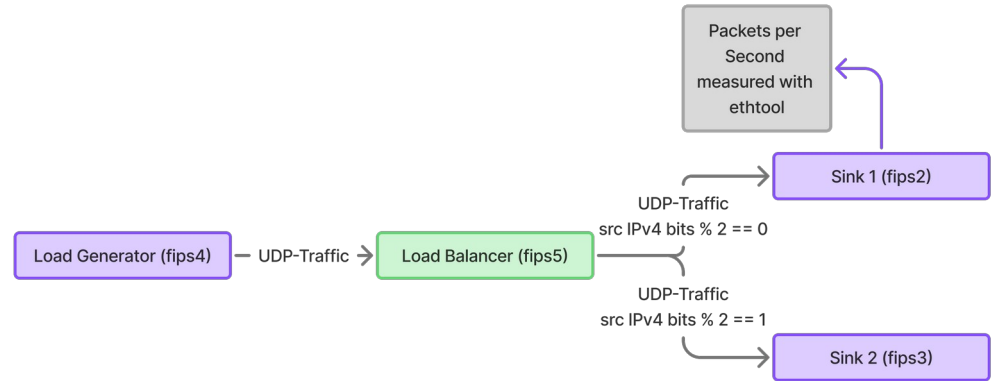
Was bisher geschah...



S. Heimbrodt, Implementierung und Evaluation eines hardwarebeschleunigten Load-Balancers auf einer Nvidia Bluefield DPU Architektur, Bachelorarbeit

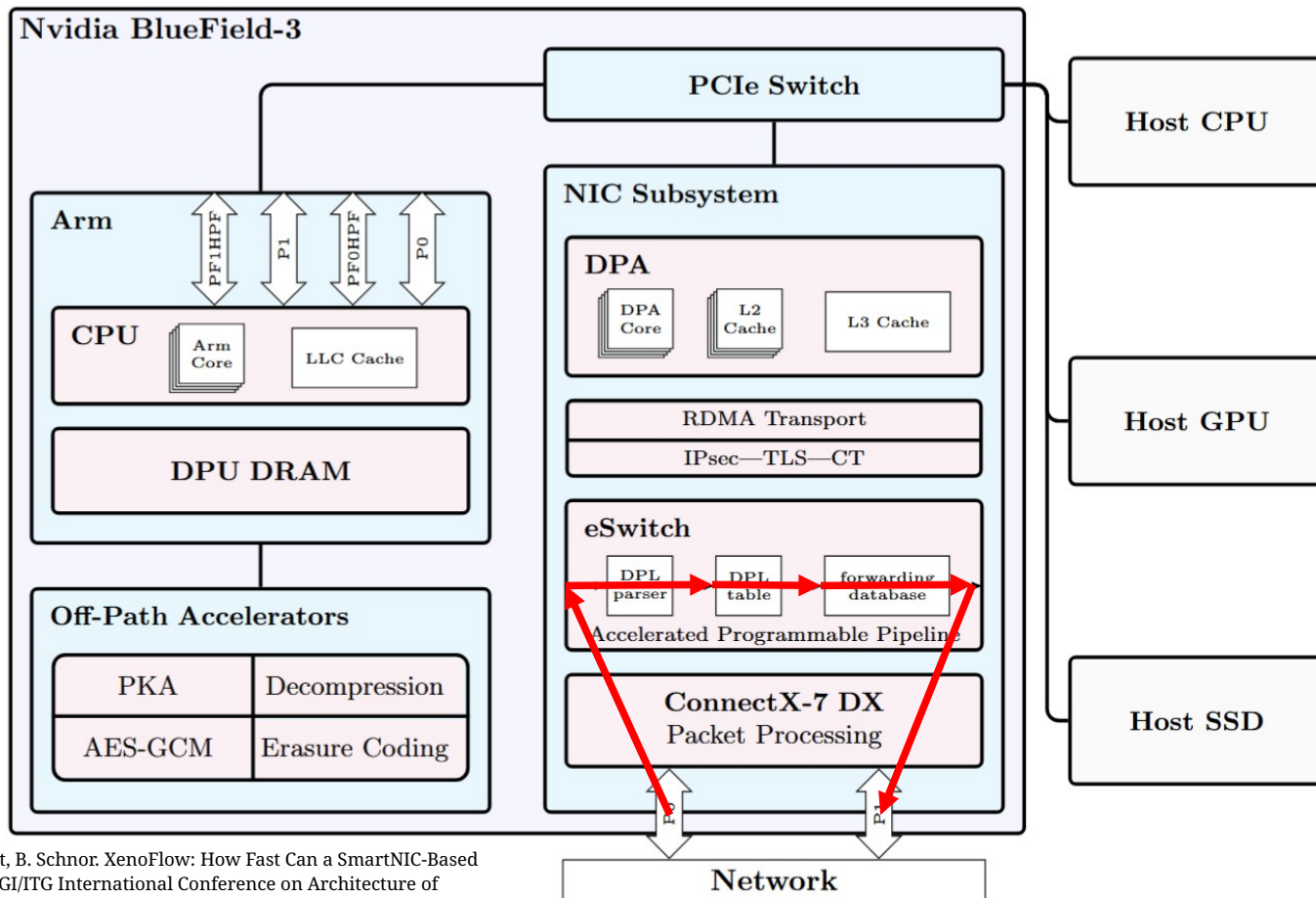
XenoFlow 0.1

- Entry pro Backend
 - keine PCC
 - kein Hinzufügen von Backends zur Laufzeit
- L4 Matching auf Source-IP
- Anfällig für Struktur die Verteilung beeinflusst



BlueField-3 Architektur

- DOCA Flow erstellt Regeln in **eSwitch**:
- **DPL Parser extrahiert Header Felder als Hierarchie**
 - Source/Destination MAC, IP, Ports, Protocol IDs
- **DPL Table** definiert Match-Action-Relation
- **Forwarding Database (FDB)** enthält eigentlich Weiterleitungsentscheidung
 - wie viele Einträge sind erlaubt?
 - wie schnell können Einträge erstellt werden?
 - Struktur?
- Vermuten den eSwitch als ASIC (FPGA?)

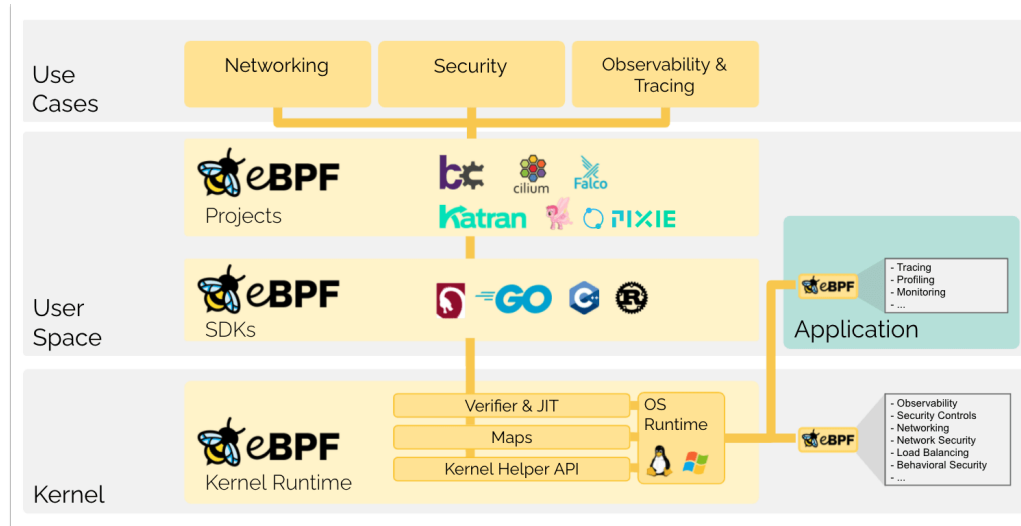


M. Schrötter, S. Heimbrodt, B. Schnor. XenoFlow: How Fast Can a SmartNIC-Based DNS Load Balancer Run? GI/ITG International Conference on Architecture of Computing Systems (ARCS) 2026

Experimente

ebpf_mac_rewrite

- Schaffung einer eigenen *Best-Case Baseline* mithilfe von eBPF
 - „run sandboxed programs in a privileged context such as the operating system kernel“
 - eBPF hooked an Events und wird verifiziert



<https://ebpf.io/what-is-ebpf/>

```

SEC("xdp")
int xdp_mac_rewrite(struct xdp_md *ctx)
{
    void *data      = (void *) (long) ctx->data;
    void *data_end = (void *) (long) ctx->data_end;

    if (!bounds_ok(data, data_end, sizeof(struct ethhdr_min)))
        return XDP_ABORTED;

    struct ethhdr_min *eth = data;

    if (!bounds_ok(data, data_end, sizeof(struct ethhdr_min)))
        return XDP_ABORTED;

    struct iphdr *ip = data + sizeof(*eth);

    if ((void *) (ip + 1) > data_end)
        return XDP_DROP;

    __be32 fips1 = 0x04030201;

    if ( __builtin_memcmp(&ip->saddr, &fips1, 4))
        return XDP_PASS;

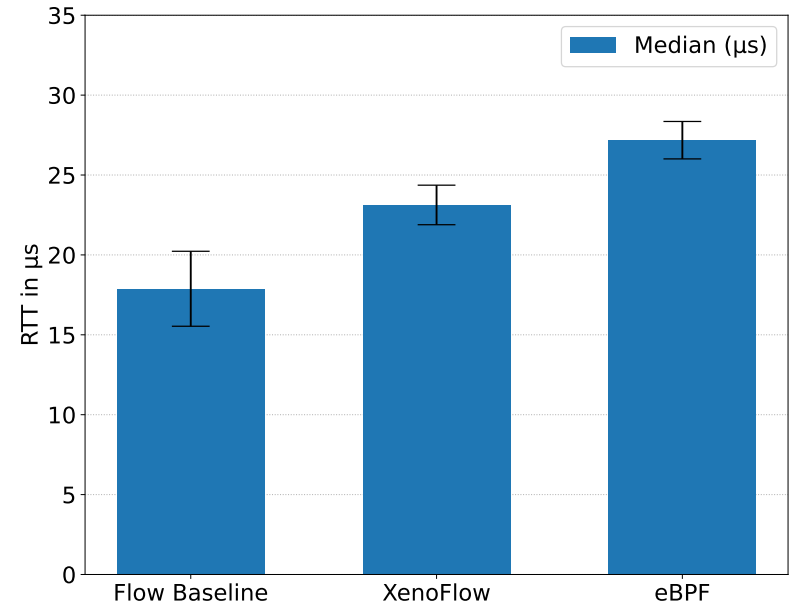
    struct mac_addr key = {};
    __builtin_memcpy(key.addr, eth->h_source, ETH_ALEN);

    const struct mac_addr *new_dst = bpf_map_lookup_elem(&mac_redirect_map, &key);
    if (!new_dst)
        return XDP_PASS;

    __builtin_memcpy(eth->h_dest, new_dst->addr, ETH_ALEN);
    //
    // New source MAC: 63:09:DE:AD:BE:EF
    // a6:03:ff:17:2b:3a
    eth->h_source[0] = 0xA6;
    eth->h_source[1] = 0x03;
    eth->h_source[2] = 0xFF;
    eth->h_source[3] = 0x17;
    eth->h_source[4] = 0x2B;
    eth->h_source[5] = 0x3A;

    return XDP_TX;
}

```



max_pipes

- Wie viele Pipes können maximal in Flow erstellt werden?

```
while(1) {  
    doca_try(result: create_root_pipe(port: ports[0], port_id: 0, out_l4_type: DOCA_FLOW_L4_TYPE_EXT_UDP, pipe: &udp_pipe), messag... "Failed to create pipe", nb_ports, ports);  
  
    struct timespec ts;  
    clock_gettime(clock_id: CLOCK_REALTIME, tp: &ts);  
  
    struct tm tm;  
    localtime_r(timer: &ts.tv_sec, tp: &tm);  
  
    DOCA_LOG_INFO(format: "Adding pipe at: %02d:%02d:%02d.%09ld", tm.tm_hour, tm.tm_min, tm.tm_sec, ts.tv_nsec);  
  
    DOCA_LOG_INFO(format: "Added nr %i", nr_pipe);  
  
    clock_gettime(clock_id: CLOCK_REALTIME, tp: &ts);  
    localtime_r(timer: &ts.tv_sec, tp: &tm);  
  
    #ifdef CLEAR  
    system("clear");  
    #endif  
    nr_pipe++;  
}
```

→ Maximal 505 Pipes

max_entries

- Wie viele Entries können maximal in Flow pro Pipe erstellt werden?

```
int nr_entry = 0;

while(1) {
    struct timespec ts;
    clock_gettime(clock_id: CLOCK_REALTIME, tp: &ts);

    struct tm tm;
    localtime_r(timer: &ts.tv_sec, tp: &tm);

    DOCA_LOG_INFO(format: "Adding entry at: %02d:%02d:%02d.%09ld", tm.tm_hour, tm.tm_min, tm.tm_sec, ts.tv_nsec);

    result = doca_flow_pipe_add_entry(0, udp_pipe, &match, &actions, &monitor, DOCA_FLOW_NO_WAIT, 0, &status, &entry);
    if (result != DOCA_SUCCESS) {
        DOCA_LOG_ERR(format: "Failed to add entry: %s", doca_error_get_descr(error: result));
        return result;
    }

    result = doca_flow_entries_process(port: ports[0], pipe_queue: 0, timeout: DEFAULT_TIMEOUT_US, max_processed_entries: num_of_entries);
    if (result != DOCA_SUCCESS) {
        DOCA_LOG_ERR(format: "Failed to process entries: %s", doca_error_get_descr(error: result));
        return result;
    }

    nr_entry += 1;
    DOCA_LOG_INFO(format: "Added nr %i", nr_entry);
    clock_gettime(clock_id: CLOCK_REALTIME, tp: &ts);
    localtime_r(timer: &ts.tv_sec, tp: &tm);

#ifdef CLEAR
    system("clear");
#endif
}
```

→ Maximal 8192 Entries pro Pipe

hash_pipe

- Flow-Feature einer Hash Pipe
→ im Prinzip Funktion von XenoFlow bit matching aus der BA
- *„The doca_flow_pipe_hash API allows for creating a pipe where entries are matched by an index, which is typically the result of a hash calculation on the packet.“*
- Hash Algorithmus nicht bekannt und nur begrenzt konfigurierbar
- Felder für Hash Nutzung können über Match Feld gesetzt werden

```

for (int i = 0; i < config->numBackends; i++) {
    struct doca_flow_fwd fwd;
    struct doca_flow_actions actions;
    enum doca_flow_flags_type flags = DOCA_FLOW_WAIT_FOR_BATCH;

    memset(s: &fwd, c: 0, n: sizeof(fwd));
    memset(s: &actions, c: 0, n: sizeof(actions));

    actions.outer.eth.dst_mac[0] = config->backends[i].mac_address[0];
    actions.outer.eth.dst_mac[1] = config->backends[i].mac_address[1];
    actions.outer.eth.dst_mac[2] = config->backends[i].mac_address[2];
    actions.outer.eth.dst_mac[3] = config->backends[i].mac_address[3];
    actions.outer.eth.dst_mac[4] = config->backends[i].mac_address[4];
    actions.outer.eth.dst_mac[5] = config->backends[i].mac_address[5];

    fwd.type = DOCA_FLOW_FWD_PORT;
    fwd.port_id = 1;

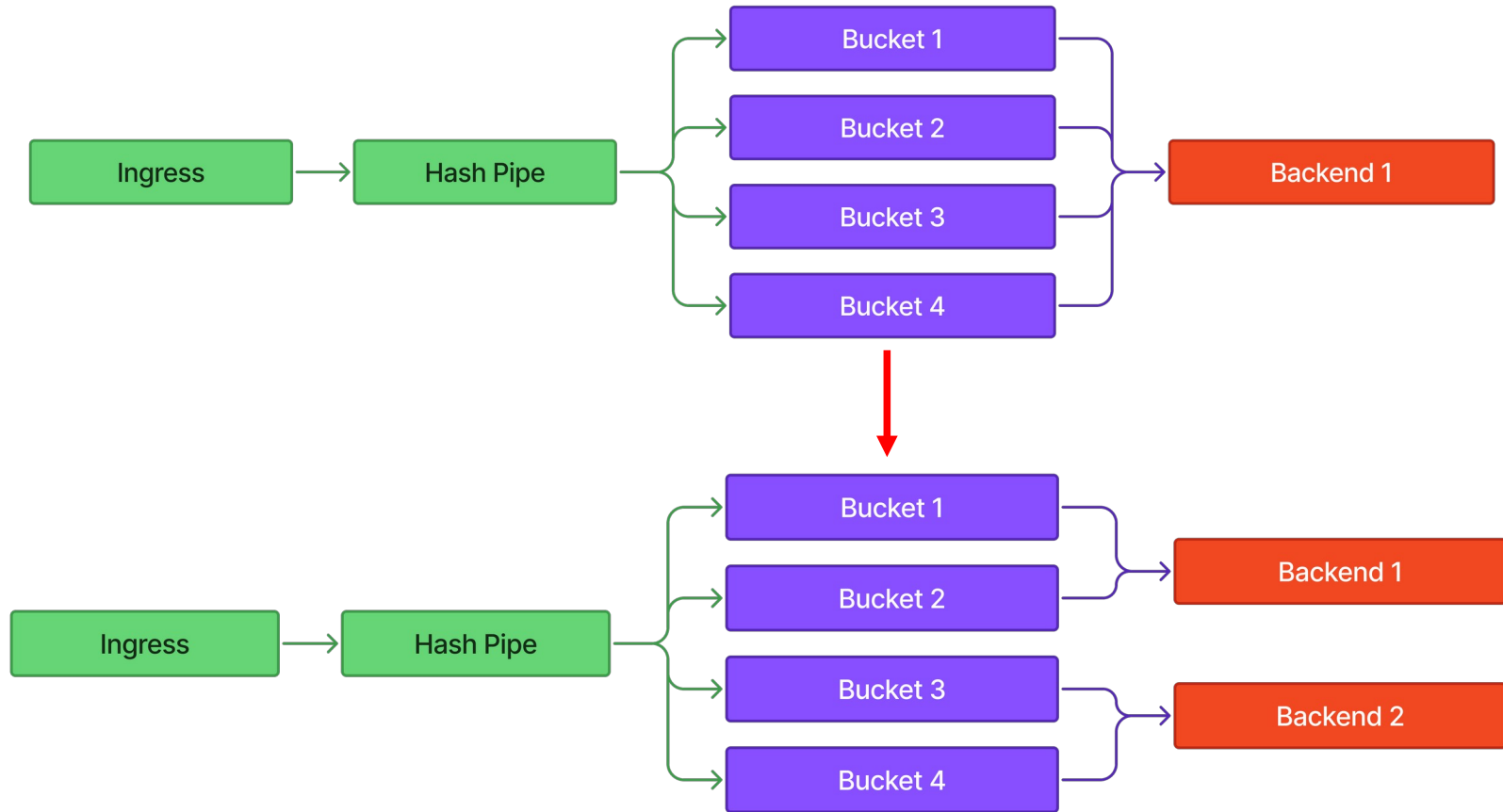
    if (i == config->numBackends - 1)
        flags = DOCA_FLOW_NO_WAIT;

    result = doca_flow_pipe_hash_add_entry(pipe_queue: 0,
        pipe: hash_pipe,
        entry_index: i,
        action_idx: 0,
        actions: NULL,
        monitor: &actions,
        fwd: &fwd,
        flags: flags,
        usr_ctx: &status,
        entry: &hash_entries[i]);
    if (result != DOCA_SUCCESS) {
        DOCA_LOG_ERR(format: "Failed to add hash entry %d: %s", i, doca_error_get_descr(error: result));
        doca_try(result, message: "Failed to add hash entry", nb_ports, ports);
    }
}

```

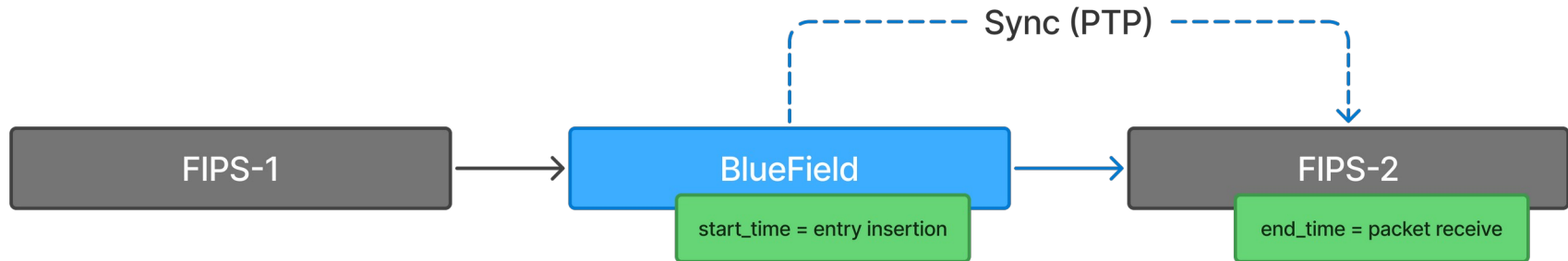

PCC

- Per-Connection-Consistency
 - Frage ob Verbindungen bei Änderungen von nicht aktivierten Feldern immernoch an das gleiche Backend geschickt werden
 - Relevant für Änderung der Anzahl an Backends
- **Bucket Semantik aus Katran** für Fully Featured Load Balancer
- Messung mittels Änderung von Feldern (Src-IP, Src-Port etc.) und Beobachtung auf Backend



entry_latency

- Wie lange braucht ein Entry bis es aktiv ist nach der Erstellung in Flow?
- Messung eines Packets im Round Trip
→ Für sinnvolle Messung: Clock Sync!
- NICs besitzen **PHC** → Sync auf Kernel → Sync zwischen Hosts



entry_latency

- Problem: PCAP File wird zu groß bei hoher Datenrate für genaue Messung
- Idee:
 - random sleep zwischen den Paketen und bilden dann Median über die *end - start timestamps*
 - Genau Messung über langen Messzeitraum

```

while(1) {
    struct timespec ts;
    int random_delay = (rand() % 10000) + 1;
    clock_gettime(clock_id: CLOCK_REALTIME, tp: &ts);

    struct tm tm;
    localtime_r(timer: &ts.tv_sec, tp: &tm);

    DOCA_LOG_INFO(format: "Adding entry at: %02d:%02d:%02d.%09ld", tm.tm_hour, tm.tm_min, tm.tm_sec, ts.tv_nsec);

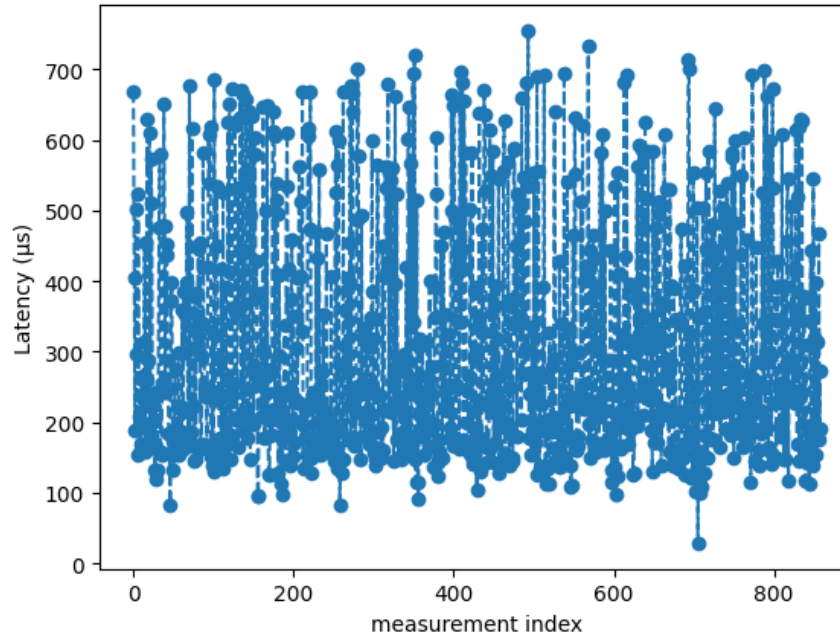
    result = doca_flow_pipe_add_entry(0, udp_pipe, &match, &actions, &monitor, DOCA_FLOW_NO_WAIT, 0, &status, &entry);
    if (result != DOCA_SUCCESS) {
        DOCA_LOG_ERR(format: "Failed to add entry: %s", doca_error_get_descr(error: result));
        return result;
    }
    result = doca_flow_entries_process(port: ports[0], pipe_queue: 0, timeout: DEFAULT_TIMEOUT_US, max_processed_entries: num_of_entries);
    if (result != DOCA_SUCCESS) {
        DOCA_LOG_ERR(format: "Failed to process entries: %s", doca_error_get_descr(error: result));
        return result;
    }
    usleep(useconds: statRefreshIntervall + random_delay);
    clock_gettime(clock_id: CLOCK_REALTIME, tp: &ts);
    localtime_r(timer: &ts.tv_sec, tp: &tm);

    DOCA_LOG_INFO(format: "Removing entry at: %02d:%02d:%02d.%09ld", tm.tm_hour, tm.tm_min, tm.tm_sec, ts.tv_nsec);
    result = doca_flow_pipe_remove_entry(pipe_queue: 0, flags: 0, entry);
    if (result != DOCA_SUCCESS) {
        DOCA_LOG_ERR(format: "Failed to remove entry: %s", doca_error_get_descr(error: result));
        return result;
    }
    result = doca_flow_entries_process(port: ports[0], pipe_queue: 0, timeout: DEFAULT_TIMEOUT_US, max_processed_entries: num_of_entries);
    if (result != DOCA_SUCCESS) {
        DOCA_LOG_ERR(format: "Failed to process entries: %s", doca_error_get_descr(error: result));
        return result;
    }
}

usleep(useconds: statRefreshIntervall);
#ifdef CLEAR
system("clear");
#endif
}

```

- BlueField PHC hat extrem hohen Jitter (32 μs)
- Verschlechterte Präzision der Messung
- 249 μs Latenz (Median) zwischen Einfügen und aktiv werden eines Entry's



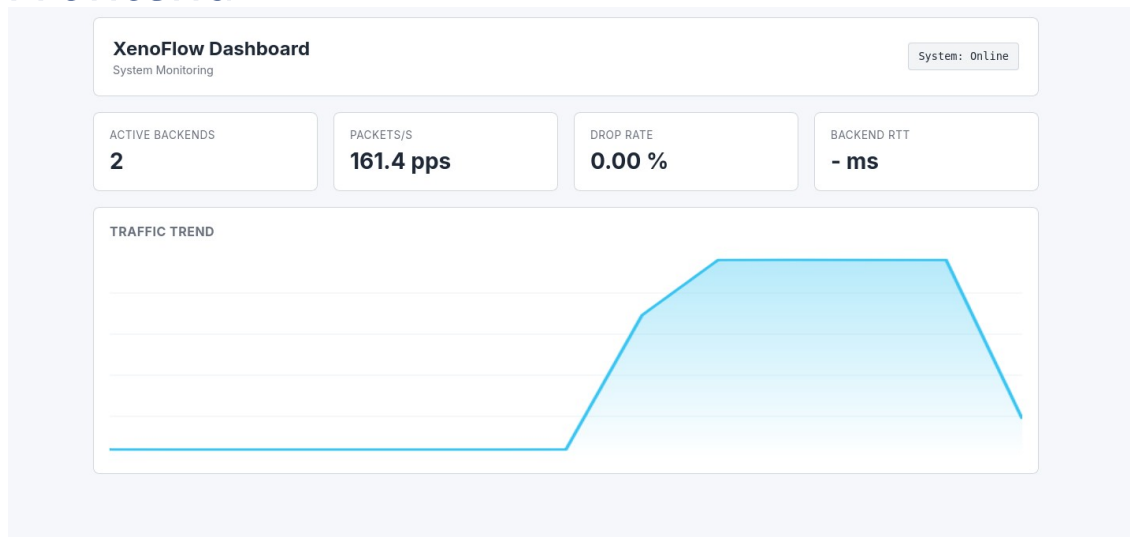
Sonstiges

- PyDFlow
 - Python bindings für DOCA Flow
 - pybind11 Framework

```
1  import pydflow
2  import time
3
4
5  # Creating a DOCA Flow Pipe with the name test and the PCIe addresses of Port 0 and Port 1 of the Bluefield
6  pyd = pydflow.PyDFlow("test", "03:00.0", "03:00.1")
7  pipe = pyd.create_pipe()
8  pipe.create_entry()
9  pipe.create_entry()
10 print(len(pipe.get_entries()))
11 print(pipe.get_entries())
12 print(pipe.get_entries()[0])
```

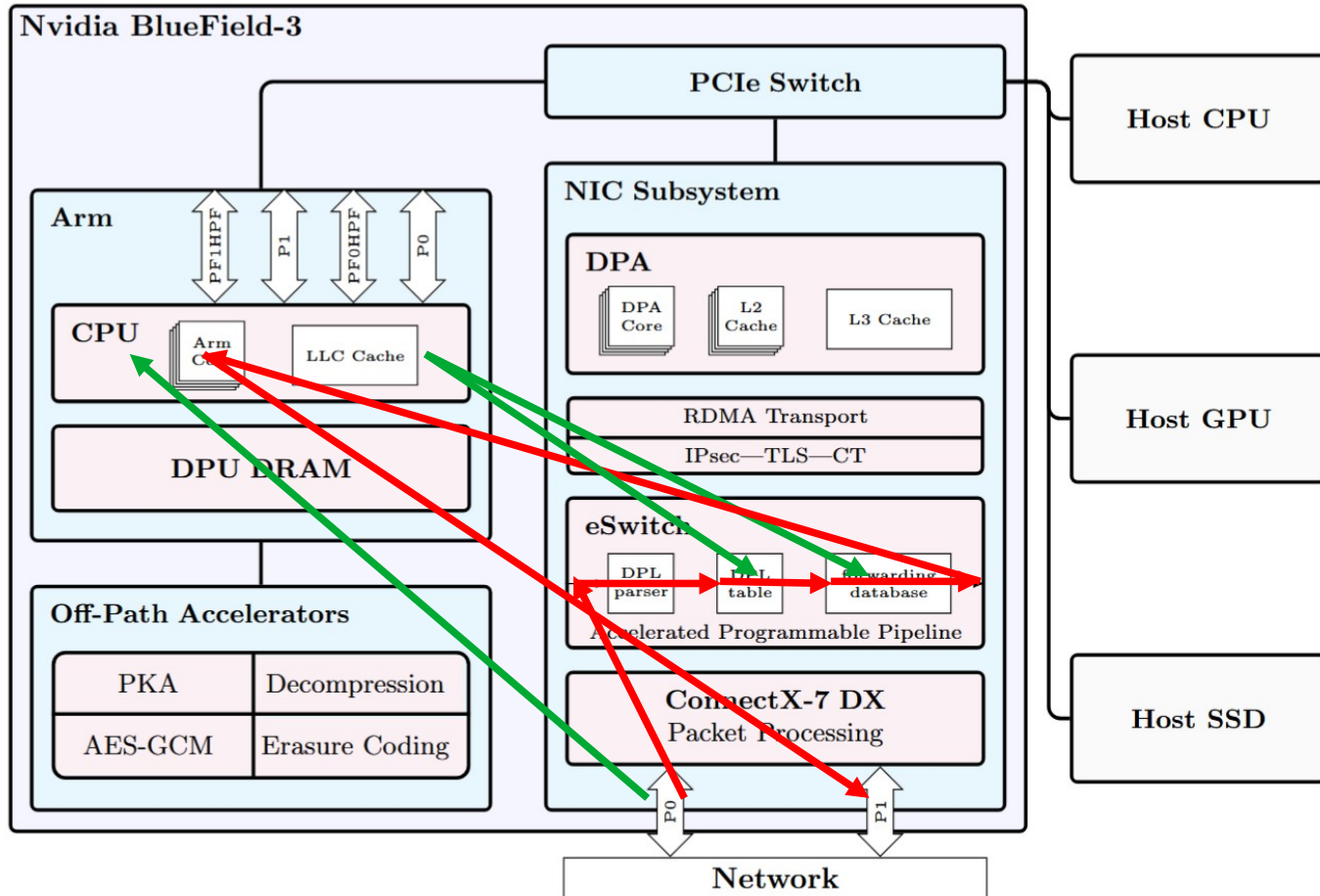
Sonstiges

- Dashboard
 - XenoFlow Rest-API zum hinzufügen und löschen von Backends
 - Flask Frontend



Ausblick

- Einhaltung von PCC mittels VeloxFlow ähnlichem Handover
 - Traffic auf veränderte Buckets auf ARM verarbeitet
 - Neue Buckets werden sofort eingetragen und „scharf“ geschaltet
 - stateful Verbindungen werden auf dem ARM sterben aus
- Alles weitere dann in VeloxFlow für QUIC
- (KubeProxy auf BlueField)



Fazit

- BlueField-3 ermöglicht viel und performantes Offloading
- *Langsame aber steile Lernkurve*
- *DPA äußerst interessant*
- Viele offene Fragen bleiben bestehen
 - Weiterhin Flow Source reverse engineering?
 - DPA Packet performance je nach Verarbeitung?
- Begrenzungen vom BF-Modell unabhängig?

Vielen Dank!

Fragen?